

Software Combining Tree for Scalable Shared Counting

Abstract

This project investigates the design, implementation, and evaluation of a scalable concurrent counter using the software combining tree technique. Traditional shared counters based on atomic operations such as fetch-and-add (FAA) or compare-and-swap (CAS) suffer from contention and scalability bottlenecks as the number of threads increases. To address this, we implemented a combining tree data structure that reduces contention by aggregating increment operations along a hierarchical tree before applying updates at the root.

The project includes a full implementation of the combining tree with configurable fan-out, correctness testing using structured test suites, and performance benchmarking against baseline approaches (atomic FAA and CAS loop). Experimental results show that while combining trees introduce higher latency at low thread counts, they demonstrate improved scalability under higher contention, especially compared to CAS-based implementations.

Our findings highlight the trade-off between latency and scalability in concurrent data structures. While atomic FAA remains the fastest under low contention, combining trees offer a viable alternative in high-contention scenarios by reducing synchronization hotspots. The project confirms that hierarchical synchronization techniques can significantly improve scalability in multi-threaded systems.

1. Goals

The primary goal of this project was to design and implement a **scalable concurrent counter** using a **software combining tree**, and to evaluate its performance against traditional synchronization mechanisms.

What was implemented?

We implemented:

- A **software combining tree counter**
- Baseline counters:
 - Atomic Fetch-and-Add (FAA)
 - CAS-based counter

Why is it interesting?

Concurrent counters are fundamental building blocks in parallel systems. However:

- Simple atomic counters become bottlenecks under contention.
- Multiple threads compete on a single memory location.
- This leads to cache coherence traffic and degraded performance.

The combining tree approach:

- Distributes contention across multiple nodes
- Combines operations before reaching the root
- Reduces synchronization overhead

Connection to Course Topics

This project relates directly to:

- Concurrent data structures
- Synchronization primitives (CAS, atomic operations)
- Scalability and contention
- Lock-free and blocking synchronization
- Parallel performance analysis

Research Question

Can a software combining tree improve scalability and reduce contention compared to traditional atomic and CAS-based counters under multi-threaded workloads?

2. Background

A **software combining tree** is a hierarchical data structure used to reduce contention in concurrent updates.

Basic Idea

Instead of all threads updating a single counter:

- Threads are assigned to leaf nodes
- Updates propagate upward
- Intermediate nodes combine operations
- Only aggregated updates reach the root

Key Concepts

1. Combining

- Multiple increments are merged

- Reduces number of atomic operations
- 2. **Tree Structure**
 - Nodes arranged in a tree
 - Each node handles a subset of threads
- 3. **Synchronization**
 - Threads synchronize at nodes
 - Use atomic primitives for coordination

Algorithm Overview

1. Thread starts at a leaf node
2. Moves upward toward root
3. Combines increments with other threads
4. Root applies final update
5. Result propagated back

Advantages

- Reduces contention
- Improves scalability
- Distributes workload

Disadvantages

- Increased latency
- More complex synchronization
- Overhead at low thread counts

3. Tasks Undertaken

3.1 Implementation

We implemented the combining tree with the following components:

Data Structures

- Tree nodes containing:
 - Status flags
 - Partial results
 - Synchronization state
- Configurable **fan-out** (branching factor)

Key Design Choices

- Fan-out values tested: **2 and 4**
- Threads mapped to leaf nodes
- Tree built based on height and thread count

Atomic Operations Used

- Compare-and-swap (CAS)
- Atomic reads/writes

Invariants

- Each node correctly combines values from children
- No lost updates
- Proper synchronization between threads

Subtle Challenges

- Avoiding race conditions
 - Maintaining correctness under high contention
 - Ensuring threads do not deadlock
-

3.2 Testing and Verification

We performed extensive testing using the provided test suite.

Test Coverage

From the logs :

- Tree construction validation
- Structural correctness
- Single-thread correctness
- Multi-thread correctness
- Stress testing
- Baseline comparisons

Key Tests

- `single_thread_basic`
- `multi_thread_small`
- `multi_thread_medium`
- `high_stress`
- `max_concurrent`

Results

- All tests passed successfully
- No correctness violations detected

Bugs Identified and Fixed

- Initial synchronization issues in node combination
- Incorrect propagation of results
- Edge cases with root node handling

Testing ensured:

- Linearizability
- Correct aggregation
- Stability under load

4. Evaluation

Experimental Setup

- Platform: Quad-core system (WSL environment)
- Threads: 2 to 8
- Workload:
 - 50,000 increments per thread
 - 5 runs per configuration
- Metrics:
 - Throughput (ops/sec)
 - Latency (ns/op)

Results Summary

Baseline Performance

- **Atomic FAA**
 - Highest throughput
 - Lowest latency
- **CAS Loop**
 - Moderate performance
 - Degrades under contention

Combining Tree Performance

- Higher latency
- Improved scalability trend

- Better than CAS at high thread counts

Key Observations

1. Low Thread Count (2–3 threads)

- FAA dominates
- Combining tree is slower due to overhead

2. Medium Thread Count (4–6 threads)

- FAA still strong
- CAS performance degrades
- Combining tree begins to scale

3. High Thread Count (7–8 threads)

Important observation:

- CAS throughput drops significantly
- Combining tree (fan=2) **outperforms CAS**
- FAA still remains best overall

Example:

- 8 threads:
 - CAS: ~4.9M ops/sec
 - Combining tree (fan=2): ~5.2M ops/sec

Fan-out Comparison

Fan = 2

- Better performance
- Lower latency
- More balanced tree

Fan = 4

- Worse performance
- Higher latency
- Increased synchronization overhead

Scalability Analysis

Method	Scalability	Latency	Best Use Case
Atomic FAA	Medium	Low	Low contention
CAS Loop	Poor	Medium	Simple systems
Combining Tree	Good	High	High contention

Interpretation

- Combining trees **reduce contention** effectively
- However, **overhead dominates at low concurrency**
- Best suited for:
 - High thread counts
 - Contention-heavy workloads

5. Conclusions

Answer to Research Question

The software combining tree **does improve scalability under contention**, particularly when compared to CAS-based counters. However, it does **not outperform atomic FAA** in this setup.

Main Findings

1. **Correctness achieved**
 - All tests passed
 - Implementation is robust
2. **Performance trade-offs**
 - Higher latency due to tree traversal
 - Reduced contention improves scalability
3. **Best configuration**
 - Fan-out = 2 performs better than fan-out = 4
4. **Comparison with baselines**

- FAA remains fastest overall
- Combining tree beats CAS at high thread counts

Limitations

- Limited to 8 threads (hardware constraint)
- WSL environment may affect performance
- No NUMA-aware optimizations
- No adaptive fan-out tuning

Future Work

If given more time, we would:

1. Test on **larger multi-core systems**
2. Implement **adaptive tree structures**
3. Explore **lock-free combining trees**
4. Optimize memory layout (cache-aware design)
5. Compare with **flat combining techniques**

Final Remark

This project demonstrates that **hierarchical synchronization techniques like combining trees are valuable tools for improving scalability**, but their effectiveness depends heavily on workload characteristics and system architecture.